



SolarChian: Updated Proposal

Motivation

Our current modification reframes the project as a physics-grounded embodied IoT testbed for urban solar coordination. In this new framing, each solar DER node is no longer treated as only a row in a dataset, a point on a map, or a contract entry. Instead, it becomes a lightweight embodied agent with a physical body, environmental perception, generation reporting behavior, verification history, trust state, and feedback memory. The blockchain component remains important, but it is no longer the main contribution. It becomes the settlement and audit layer that records approved agent actions after physical verification and planner review.

Refactoring Direction

The central change is the addition of a persistent agent layer between the simulator, frontend, and smart contracts. Previously, the simulator generated solar and market data, the frontend displayed verification evidence, and the contract handled settlement. These pieces were useful but loosely connected. The new design connects them through an event-driven coordination loop.

In this loop, a `SolarAgent` persists across time. It observes weather and generation context, estimates a physics-bounded expected output, submits a reported generation value, receives machine verification and planner judgment, participates in market coordination if approved, and updates its trust score and calibration state after feedback. This persistent state is essential: it prevents the agent layer from becoming a superficial wrapper over CSV records. A solar node now has memory, reputation, and a changing participation state.

The planner and market components were also repositioned. The planner is no longer only a frontend review metaphor; it is represented as a decision-making agent that can approve, reject, or escalate reports based on verification results and trust. The market layer no longer assumes a fixed reward/liquidity split as a hard-coded result. Instead, it supports configurable split ratios so we can study the tradeoff between producer incentives, available liquidity, and market slippage.

Main Modifications

A monthly episode can now run over the five-city dataset, preserving per-agent state over time and producing persistent outputs such as event logs, audit logs, agent decisions, market summaries, and state histories.

Every major system action is recorded as an event: solar report, verification result, planner decision, market update, and feedback update. These events include a hash chain through `record_hash` and `previous_hash`, which gives the system an auditable trace of how physical reports become coordination and settlement decisions. This supports the EloT safety and verifiability angle much better than a session-only frontend log.

We also connected the agent layer to the existing FastAPI simulator. The backend can now initialize an agent episode, step through the coordination loop, return recent events, expose audit records, summarize market feedback, and attempt settlement of approved reports. This makes the agent layer accessible to the frontend and prepares the system for connected demonstrations rather than only offline experiments.

On the frontend, we added panels for agent trace and coordination state. These panels show recent agent events, verification outcomes, planner actions, market feedback, settlement hashes, and trust updates. This is important because it makes the new research story visible: a user can inspect not only whether a report was accepted, but how the decision was produced and how it affected the agent afterward.

The smart contract was also refactored. The previous 25/75 reward-liquidity split is now configurable through `rewardRatioBps`, with tests covering the default ratio, owner updates, invalid updates, and settlement behavior under a changed ratio. This change directly addresses a likely reviewer concern: the system should not present one fixed economic split as if it were naturally optimal. Instead, the split becomes an experimental variable.

Finally, we added a set of EloT-oriented experiments. The current experiment suite covers false-data attack taxonomy, adaptive verification, reward/liquidity ratio sweep, system overhead, and scale benchmarking. These experiments help us evaluate the system as a benchmark and coordination testbed, rather than only as a visual prototype.